

Banco de voz pregenerado: ¿funciona?

Actualizado: 09/08/2026. Análisis de la idea de pregenerar un corpus grande de audio y sintetizar en vivo solo lo que falte. Todos los números vienen de medidas en A100 SXM con el motor actual, no de estimaciones.

Veredicto en tres líneas

Sí funciona, pero no con palabras y no con frases al azar. La unidad tiene que ser la **frase completa** — encadenar palabras sueltas suena a contestador de los noventa— y el corpus tiene que salir de una **gramática enumerada**, no de un montón de frases sueltas.

Si se acepta que el agente hable dentro de esa gramática, el acierto es del **100% por construcción** y el TTS sale del runtime entero: sin GPU, sin límite de llamadas simultáneas, 590 ms por turno. Ese camino está en la sección 8.

1. Por qué las palabras no valen

Un TTS moderno no genera palabras, genera una curva de entonación sobre la frase entera. La misma palabra suena distinta según dónde caiga:

- "**Perfecto.**" — tono descendente, cierre.
- "¿**Perfecto?**" — ascendente, pregunta.
- "**Perfecto**, entonces..." — suspendido, continúa.

Si guardas "perfecto" una vez y la reutilizas, dos de cada tres veces suena mal. Y al concatenar aparecen los cortes de coarticulación: los fonemas del final de una palabra se moldean con el principio de la siguiente, y esa transición no existe si vienen de generaciones distintas.

Es exactamente el sonido de "su... saldo... es... de... veintitrés... euros". Técnicamente funciona, comercialmente destruye la llamada.

La unidad mínima cacheable es la frase entre puntos. Es donde el modelo cierra la entonación y donde el motor ya trocea hoy.

2. La cuenta que importa: acierto contra latencia

Medido: un turno cuesta **~590 ms** si la primera frase está cacheada y **~1.600 ms** si hay que sintetizarla (una frase corta como "Perfecto." son 1,20 s de audio, que a 1,19x tiempo real cuesta 1.010 ms).

$$\text{latencia media} = 590 \text{ ms} + (1 - \text{acierto}) \times 1.010 \text{ ms}$$

Acierto de caché	Latencia media
0%	1.600 ms
50%	1.095 ms
70%	893 ms
90%	691 ms ← el objetivo
100%	590 ms

Hace falta un 90% de acierto sobre primeras frases para que la media caiga por debajo de 700 ms. Ese es el número que dirige todo el diseño.

3. Por qué 10.000 frases no dan el 90%

Una conversación usa ~2.000 palabras, pero la pregunta no es cuántas palabras hay: es **cuántas frases distintas** pueden salir. Con un vocabulario de 2.000 palabras y frases de 8 a 15, el espacio de frases posibles no es grande, es infinito a efectos prácticos.

Y hay frases que **jamás** estarán en el banco porque contienen datos del cliente concreto:

"Perfecto, señor **Iriarte**, le confirmo la cita para el **jueves 14** a las **cinco y media** en **Valencia**, y son **doscientos noventa** euros."

Pregenerar 10.000 frases al azar tiene un acierto bajísimo porque estarías cubriendo el espacio equivocado. La curva se aplana enseguida:

Corpus	Acierto estimado sobre primeras frases
50 fórmulas	~45%
300 fórmulas	~65%
5.000 frases	~70%
10.000 frases	~72%

De 300 a 10.000 ganas siete puntos. No es un problema de tamaño.

4. El diseño que sí llega al 90%

Dos cambios de enfoque, ninguno de volumen.

4.1 Cachear solo la primera frase

Lo demás no hace falta. Mientras suena la primera frase (~1,2 s de audio), el motor sintetiza la segunda a 1,19x tiempo real: llega antes de que se acabe la primera. El TTS solo está en el camino crítico una vez por turno.

Eso reduce el problema de "cachear una conversación" a "cachear los arranques", que es un conjunto **cerrado y pequeño**.

4.2 Que Node elija la apertura, no el LLM

Hoy el prompt *pide* al modelo que abra corto. Pedir no es garantizar: si el modelo improvisa "Muy bien, pues mire," en vez de "Perfecto.", hay fallo de caché y el turno cuesta 1.600 ms.

La directiva del policy gate ya lleva `nextAction`. Node puede **elegir la apertura de un catálogo cerrado** según esa acción y anteponerla, dejando al LLM solo el contenido:

```
Node decide:  nextAction = HANDLE_OBJECTION
Node elige:    "Entiendo."          (del catálogo, ya en caché → 0 ms)
LLM escribe:  "El precio incluye las llamadas ilimitadas..."
```

Con eso el acierto sobre primeras frases **es del 100% por construcción**, no por estadística. Y de paso da variedad controlada: rotar entre 5 aperturas por cada acción evita que el agente suene a robot repitiendo "Perfecto".

Catálogo necesario: 9 acciones comerciales × ~~6 variantes × 6 registros de voz~~ = ****320 frases****. Ese es el número correcto, no 10.000.

5. Lo que cuesta

Nada, y ese es el argumento más fuerte para hacerlo.

	320 frases	10.000 frases
Tiempo de GPU (1,26 s por frase)	7 min	3,5 h
Coste en A100 a 1,62 \$/h	0,19 \$	5,67 \$
Disco (24 kHz, 16 bits, ~1,5 s)	23 MB	720 MB
Lectura por acierto (NVMe)	<1 ms	<1 ms

Ni el tiempo ni el espacio son el límite. Pregenerar es barato; lo caro es elegir mal qué pregenerar.

6. Los tres problemas reales

6.1 La costura entre frases

La frase cacheada y la generada en vivo salen de ejecuciones distintas, y el modelo muestrea con `temperature = 0.9`: dos tomas de la misma voz tienen prosodia distinta. Al empalmarlas puede notarse el salto.

Mitiga bastante que el corte esté en un **límite de frase**, que es donde una persona también reinicia la entonación. Refuerzos posibles: bajar la temperatura del TTS, fijar semilla, y un fundido de 20-30 ms en la unión.

Esto hay que escucharlo antes de dar el diseño por bueno. Es el único riesgo que ningún número resuelve.

6.2 El banco caduca con la voz

Las claves del caché ya incluyen voice pack, versión y registro, así que un cambio de voz no sirve audio equivocado — simplemente falla y regenera. Pero regregar el actor invalida las 320 frases y hay que volver a pregenerar. Siete minutos, asumible.

6.3 Sigue sin haber margen para concurrencia

A 1,19x tiempo real, dos llamadas simultáneas vuelven a bajar de tiempo real y el audio se corta. El caché no lo arregla: solo quita la primera frase del camino crítico, no acelera las siguientes. Eso sigue necesitando el bucle exterior reescrito o una GPU más rápida.

7. Qué implementar, en orden

1. **Catálogo de aperturas por acción comercial** (~320 frases) y script que lo sintetice offline contra cada voice pack.
2. **Caché persistente en disco**, con clave por voice pack y versión. Hoy vive en memoria y muere al reiniciar el motor: la primera llamada tras cada arranque paga el TTS entero.
3. **Node elige la apertura** desde la directiva, en vez de pedírsela al prompt.
4. **Medir el acierto real** con el campo `cached` de `tts.first_audio`. Si no sube del 90%, el catálogo o la elección de Node están mal, no el tamaño.
5. **Escuchar diez llamadas** buscando la costura. Si canta, bajar temperatura y añadir el fundido.

Los pasos 1 a 3 son medio día de trabajo y valen ~1.000 ms por turno. Es la mejor relación esfuerzo/latencia que queda en el sistema.

8. La vía del 100%: gramática enumerada

Todo lo anterior asume que el LLM escribe texto libre y el caché intenta adivinarlo. Si el objetivo es el 100%, hay que invertir la relación: **el agente solo puede decir frases que existan en el banco**, y el banco se genera enumerando una gramática cerrada.

No es cachear más. Es cambiar qué dice el agente.

8.1 Por qué esto sí cierra el 100%

Lo que impedía llegar al 100% eran las frases con datos del cliente. Pero esos datos **no son infinitos, son enumerables**:

Hueco	Valores posibles	Enumerable
Día de la cita	lunes-domingo, mañana, pasado	9
Hora	8:00-20:00 cada 30 min	25
Importe	los planes de la base de conocimiento	3-10
Duración, plazo	catálogo del negocio	~10
Nombre del cliente	infinito	no

Así que "le confirmo la cita para el jueves a las cinco y media" no es una frase: son $9 \times 25 = \mathbf{225}$ frases **completas**, todas pregenerables. Se cachea la frase entera con el dato dentro, nunca se empalma el dato por separado — y así no hay costura ni entonación rota.

El nombre propio es la única excepción real. Se resuelve no metiéndolo en la primera frase: "Perfecto, le confirmo la cita..." funciona igual de bien que "Perfecto, señor Iriarte, le confirmo...".

8.2 Tamaño real de la gramática

Bloque	Frases	× registros	Total
Aperturas por acción comercial	54	6	324
Confirmación de cita (día × hora)	225	2	450
Propuesta de hueco (día × hora)	225	2	450
Precios y condiciones	30	2	60
Objeciones estándar	36	3	108
Preguntas de descubrimiento	40	3	120
Aviso de IA, consentimiento, opt-out	12	3	36
Cierres y despedidas	20	4	80
Muletillas y backchannel	15	6	90
Total			~1.700

Con cinco variantes de redacción por nodo —para que no suene a bucle— se va a **~8.500 frases**. Sigue siendo pequeño.

8.3 Lo que cuesta generarlo

	1.700 frases	8.500 frases
GPU (1,26 s por frase, A100)	36 min	3 h
Coste a 1,62 \$/h	0,97 \$	4,86 \$
Disco (24 kHz, 16 bits)	122 MB	612 MB

Cinco euros y una tarde de GPU, **una sola vez por voz**.

8.4 La consecuencia grande: el TTS sale del runtime

Con acierto del 100%, en una llamada no se sintetiza nada. Y eso cambia la infraestructura entera:

	Hoy (síntesis en vivo)	Con gramática al 100%
GPU para TTS	A100, 1,62 \$/h	ninguna
Llamadas simultáneas	1	sin límite práctico
Latencia del TTS	1.010 ms	<1 ms (leer de disco)
Turno completo	~1.600 ms	~590 ms
Riesgo de que invente un precio	existe	cero

La GPU solo seguiría haciendo falta para el STT, que es whisper `small` y cabe en cualquier tarjeta barata — o en CPU si hace falta.

Ese último punto no es de latencia sino de cumplimiento, y es el más valioso: **cada palabra que dice el agente está pregenerada y revisada**. No puede inventarse un precio ni un plazo porque no existe el audio para decirlo. Para el artículo 50 del Reglamento de IA y para cualquier auditoría, eso es una posición mucho más fuerte que "el prompt le dice que no invente".

8.5 Lo que se pierde

El agente deja de improvisar. Si el cliente pregunta algo que no está en la gramática, no hay frase que responder.

Salidas, por orden de preferencia:

1. **Ampliar la gramática.** Cada llamada real que caiga fuera es un nodo nuevo que se añade y se pregenera esa noche. Al mes, la cobertura sube sola.
2. **Frase puente cacheada** — "Déjeme confirmarlo con un compañero." — y transferir. Es lo que haría un comercial junior, y es honesto.
3. **Síntesis en vivo** para ese turno: 1,6 s de latencia puntual. Sigue estando disponible, solo deja de ser el caso normal.

Para una llamada comercial con playbook esto no es una limitación real: un comercial humano también trabaja con un guion. Para atención al cliente abierta sí lo sería.

8.6 Qué cambia en el código

- 1. **Definir la gramática** como datos (JSON por vertical), no en el prompt.
- 2. **Script de enumeración y síntesis** offline: recorre la gramática, sintetiza cada hoja contra cada voice pack y escribe el banco a disco.
- 3. **Caché en disco** en el motor, con la clave que ya existe (voice pack, versión, registro, texto normalizado).
- 4. **Node selecciona el nodo de la gramática** a partir de `nextAction` y del estado comercial; el LLM pasa de redactar a **elegir** entre las opciones válidas y rellenar huecos enumerados.
- 5. **Contador de fallos**: cada texto que no esté en el banco se registra. Esa lista es la cola de trabajo de la gramática.

Los puntos 1 a 3 son independientes y ya valen por sí solos (bajan la latencia sin tocar la conversación). El 4 es el que exige decidir que el agente habla con guion cerrado.

9. Estado de la implementación

Hecho y medido el 09/08/2026 en A100 SXM:

Pieza	Estado
Catálogo cerrado (<code>aperturas.json</code>)	41 microintenciones, 287 textos, 814 audios
Banco pregenerado	814/814 · 47 MB · 12,8 min · ~0,35 \$ · cobertura 100% verificada
Banco en disco en el motor (<code>voice_bank.py</code>)	Hecho; sobrevive a reinicios
Selector en Node (<code>openings.ts</code>)	Hecho, con cascada de respaldo y objeción fina
Apertura en la directiva y hablada antes del LLM	Hecho en telefonía y simulador
Prueba telefónica real	Pendiente

El motor habla la apertura **antes** de llamar al LLM, así que del presupuesto del turno desaparecen dos tramos, no uno:

```
fin de turno 250 + STT 100 + gate 40 + apertura del banco ~0 = ~390 ms
```

El LLM ya no está en el camino crítico: tiene toda la duración de la apertura (0,6-1,9 s) para producir su primera frase de contenido.

Lo que hay que vigilar en la primera prueba real:

- **Que las dos versiones del voice pack coincidan.** El banco se generó con `actor_v1@1.0.0`; si `backend/.env` envía otra, el acierto es 0%.
- **La costura** entre la apertura pregenerada y la primera frase en vivo (sección 6.1). Es lo único que ningún número resuelve.
- **El campo `cached`** de `tts.first_audio` y el evento `turn.opening_spoken`: ahí se lee el acierto real.

10. Base

Medidas en A100 SXM 80 GB, Qwen3-TTS 12Hz 1.7B CustomVoice con `qwen_fast.py` activo: 1,19-1,40x tiempo real (0,57x sin él). Caché de audio y troceado por frases: `server.py`. Presupuesto del turno: `sistema-llamadas.md`.